DSA STUDY BLUEPRINT SERIES

102. BST Iterator

Inorder Traversal Iterations using Stacks

Section 09 • C++ Core Data Structures & Algorithms

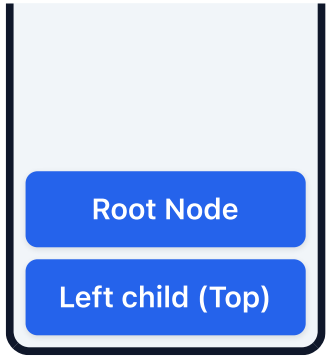
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **BST Iterator:** Supports next() and hasNext() in O(1) time.
- Push left branch nodes onto stack; pop, store, and process right branches.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
class BSTIterator {
    stack st;
    void pushLeft(TreeNode* node) {
        while(node) { st.push(node); node = node->left; }
    }
public:
    BSTIterator(TreeNode* root) { pushLeft(root); }
    int next() {
        TreeNode* node = st.top(); st.pop();
        pushLeft(node->right); return node->val;
    }
};
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
next() Operation	O(1) amortized
Space Complexity	O(H) (Stack elements)

Key Practices & Gotchas

- **Important:** Amortized O(1) runtime since each node is pushed and popped at most once.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	[-∞, +∞]	True (-∞ < 15 < +∞)	Recurse Left child with range [-∞, 15]
10 (Left)	[-∞, 15]	True (-∞ < 10 < 15)	Recurse Right child with range [10, 15]

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.